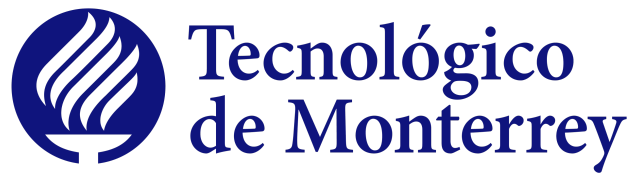


Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey



TE3001B

Fundamentación de robótica

Mini-reto: Generar nodo que envíe una señal a otro nodo que procese la señal

Alumnos:

Josue Ureña Valencia	IRS A01738940
César Arellano Arellano	IRS A00839373
Jose Eduardo Sanchez Martinez	IRS A01738476
Rafael André Gamiz Salazar	IRS A00838280

Docente:

Luis Alberto Muñoz Ubando

Fecha de entrega:

18 de febrero del 2026

Código implementado para el nodo “signal_gen”

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
import math
import time

#Class Definition
class SignalGenerator(Node):
    def __init__(self):
        super().__init__('signal_gen')
        #Write your code here
        self.signal_publisher =
self.create_publisher(Float32, '/signal', 10)
        self.time_publisher = self.create_publisher(Float32, '/time',
10)

        timer_period = 0.1 #0.1 seconds -> 10 Hz
        self.timer = self.create_timer(timer_period, self.timer_cb)
        self.start_time = time.time()

        #Timer Callback
    def timer_cb(self):
        t = time.time() - self.start_time # Tiempo t
        signal = math.sin(t) #señal
        msg_signal = Float32()
        msg_time = Float32()

        msg_signal.data = float(signal)
        msg_time.data = float(t)

        self.signal_publisher.publish(msg_signal)
        self.time_publisher.publish(msg_time)
        self.get_logger().info(f"Time: {t:.2f} | Signal: {signal:.2f}")

def main(args=None):
    rclpy.init(args=args)
    signal_generator = SignalGenerator()
    try:
        rclpy.spin(signal_generator)
    except KeyboardInterrupt:
        pass
    finally:
```

```

        if rclpy.ok(): #Ensure shutdown is only called once
            rclpy.shutdown()
            signal_generator.destroy_node()

#Execute Node
if __name__ == '__main__':
    main()

```

Nodo 1: Generador de señal (signal_gen)

El primer nodo implementa un generador de señal en ROS 2 utilizando rclpy. Su función principal es producir una señal sinusoidal en tiempo real y publicarla periódicamente en un tópico para que otro nodo pueda suscribirse y procesar dicha señal.

Al iniciar, el nodo se crea con el nombre “signal_gen” y configura dos publicadores con mensajes del tipo Float32, el tópico “/signal”, donde se publica el valor instantáneo de la señal, y el tópico “/time”, donde se publica el tiempo correspondiente. Para controlar la frecuencia de publicación, el nodo utiliza timer con un periodo de 0.1 segundos, equivalente a la frecuencia solicitada de 10 Hz.

En cada ejecución del callback del timer (timer_cb), el nodo calcula el tiempo t en el que se encuentra para posteriormente obtener el valor de la señal: $\text{signal}(t) = \sin(t)$. Posteriormente, ambos valores (t y signal) se empaquetan en mensajes Float32 y se publican en sus respectivos tópicos (/time y /signal). Adicionalmente, el nodo imprime en consola el tiempo y el valor de la señal con fines de monitoreo y depuración, permitiendo verificar que el sistema está generando y publicando datos correctamente durante la ejecución.

Finalmente, el programa mantiene la ejecución del nodo mediante rclpy.spin(), y en caso de interrupción (por ejemplo, Ctrl+C), realiza un cierre seguro llamando a rclpy.shutdown() y destruyendo el nodo con destroy_node().

Código implementado para el nodo “signal_proc”

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
import math
import time

#Class Definition
class SignalProcessing(Node):
    def __init__(self):
        super().__init__('signal_proc')
        #Variables para guardar datos recibidos
        self.signal = 0.0
        self.time = 0.0

```

```

        self.phase_shift = math.pi / 2 #Desfase (hardcoded) 90°

        self.signal_subscription = self.create_subscription(
            Float32, '/signal', self.signal_callback,10)

        self.time_subscription = self.create_subscription(
            Float32, '/time', self.time_callback,10)

        #Publisher
        self.process_publisher =
self.create_publisher(Float32,'/proc_signal',10)
        timer_period = 0.1 #0.1 seconds -> 10 Hz
        self.timer = self.create_timer(timer_period, self.timer_cb)

    def signal_callback(self, msg): #Callback del signal
        self.signal = msg.data
    def time_callback(self, msg): #Callback del time
        self.time = msg.data
    def timer_cb(self): #Callback de procesamiento (10 Hz)
        sin_t = self.signal
        cos_t = math.cos(self.time)

        shifted_signal = (sin_t * math.cos(self.phase_shift) + cos_t *
math.sin(self.phase_shift))
        offset_signal = shifted_signal + 1.0
        processed = 0.5 * offset_signal

        #Publicar resultado
        msg = Float32()
        msg.data = float(processed)
        self.process_publisher.publish(msg)
        #Imprimir
        self.get_logger().info(f"Time: {self.time:.2f} | Proc_Signal:
{processed:.2f}")

#Main
def main(args=None):
    rclpy.init(args=args)
    signal_processing = SignalProcessing()
    try:
        rclpy.spin(signal_processing)
    except KeyboardInterrupt:
        pass

```

```

    finally:
        if rclpy.ok(): #Ensure shutdown is only called once
            rclpy.shutdown()
            signal_processing.destroy_node()
#Execute Node
if __name__ == '__main__':
    main()

```

Nodo 2: Procesamiento de señal (signal_proc)

El segundo nodo implementa la etapa de procesamiento en ROS 2 utilizando rclpy. Su objetivo es recibir la señal generada por el nodo anterior, aplicarle un desfase (phase shift) y posteriormente realizar un corrimiento (offset) y un escalamiento (scaling) para obtener una señal final normalizada, la cual se publica en un nuevo tópico.

Al iniciar, el nodo se crea con el nombre “signal_proc” y se suscribe a dos tópicos de tipo Float32: “/signal”, que contiene el valor de la señal original (en este caso, una senoide), y “/time”, que contiene el tiempo correspondiente. Los valores recibidos se almacenan en variables internas (self.signal y self.time) mediante callbacks (signal_callback y time_callback), de modo que el nodo siempre dispone de la muestra más reciente para procesarla.

Posteriormente, el nodo define un parámetro fijo (hardcoded) llamado phase_shift, con valor $\phi = \pi/2$ (90°), y configura un publicador al tópico /proc_signal. El procesamiento se ejecuta a una frecuencia de 10 Hz mediante un timer con periodo de 0.1 s, garantizando que el nodo procese y publique datos continuamente. En cada ejecución del callback del timer (timer_cb), el nodo aplica un desfase utilizando la siguiente identidad trigonométrica:

$$\sin(t+\phi) = \sin(t)\cos(\phi) + \cos(t)\sin(\phi)$$

En el código, se toma $\sin(t)$ desde el valor recibido en “/signal” y se calcula $\cos(t)$ a partir del tiempo recibido en “/time”. Con esto, se obtiene una señal desfasada (shifted_signal) en $\pi/2$ radianes, este desfase convierte la senoide en un coseno: $\sin(t+\pi/2) = \cos(t)$.

Después del desfase, la señal se ajusta para obtener una salida normalizada en el rango [0,1]. Primero se aplica un offset sumando 1 para desplazar el rango de [-1,1] a [0,2]. Posteriormente, se aplica un escalamiento multiplicando por 0.5 para que al final la señal esté en el rango de [0,1].

El valor resultante se encapsula en un mensaje Float32 y se publica en el tópico “/proc_signal”. Adicionalmente, el nodo imprime en consola el tiempo y el valor de la señal procesada con un formato similar al nodo “signal-gen”, lo cual facilita la verificación del funcionamiento y el seguimiento de la ejecución en tiempo real. Al igual que el primer nodo, el programa mantiene su ejecución mediante rclpy.spin() y, en caso de interrupción, realiza un cierre seguro con rclpy.shutdown() y la destrucción del nodo con destroy_node().

Código Setup.py

```
from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'signal_processing'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*launch.[pxy][yma]*'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ed',
    maintainer_email='ed@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'signal_gen = signal_processing.signal_gen:main',
            'signal_proc = signal_processing.signal_proc:main'
        ],
    },
)
```

En el código setup.py se agregaron dos líneas para integrar correctamente el paquete en ROS 2: la línea 15 con `glob(os.path.join('launch', '*launch.[pxy][yma]*'))` se añadió para que, al compilar e instalar con `colcon`, los archivos dentro de la carpeta `launch/` también se copien a `share/<paquete>/launch` y así puedan ejecutarse con `ros2 launch`; y la línea 32 (`'signal_proc = signal_processing.signal_proc:main'`) se incluyó en `console_scripts` para registrar el segundo nodo como ejecutable del paquete y poder correrlo directamente con `ros2 run signal_processing signal_proc`, al igual que el nodo `signal_gen`.

Código del Launch

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    signal_gen_node = Node(package='signal_processing',
                           executable='signal_gen',
                           output='screen'
                           )

    signal_proc_node = Node(package='signal_processing',
                            executable='signal_proc',
                            output='screen'
                            )

    rqt_node = Node(
        name='rqt_plot',
        package='rqt_plot',
        executable='rqt_plot',
        arguments=['/signal/data', '/proc_signal/data'],
        output='screen'
    )

    l_d = LaunchDescription([signal_gen_node, signal_proc_node, rqt_node])
    return l_d
```

Este código launch file lo usamos para arrancar varios nodos a la vez con un solo comando. En `generate_launch_description()` se crean tres acciones `Node` que es `signal_gen_node`, que ejecuta el nodo `signal_gen` del paquete `signal_processing` para generar y publicar la señal y el tiempo; `signal_proc_node`, que ejecuta `signal_proc` del mismo paquete para suscribirse a `/signal` y `/time`, procesar la señal y publicar el resultado en `/proc_signal`; y `rqt_node`, que abre `rqt_plot` y le pasa como argumentos los tópicos `/signal/data` y `/proc_signal/data` para graficar en tiempo real tanto la señal original como la procesada. Finalmente, los tres nodos se agrupan dentro de un `LaunchDescription` y se regresan para que al ejecutar `ros2 launch` se inicien simultáneamente.

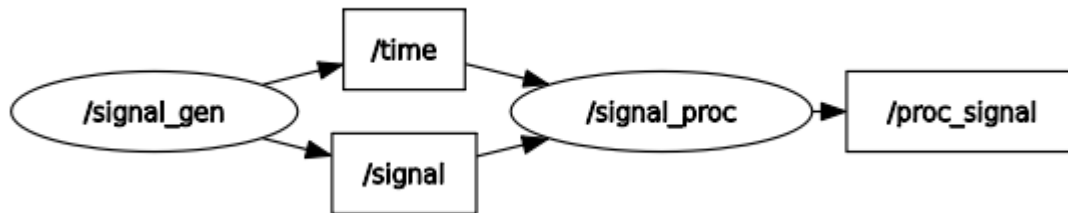
Finalmente en la terminal se ejecutan los siguiente comandos y podemos observar como los nodos estan funcionando.

```
$ colcon build
```

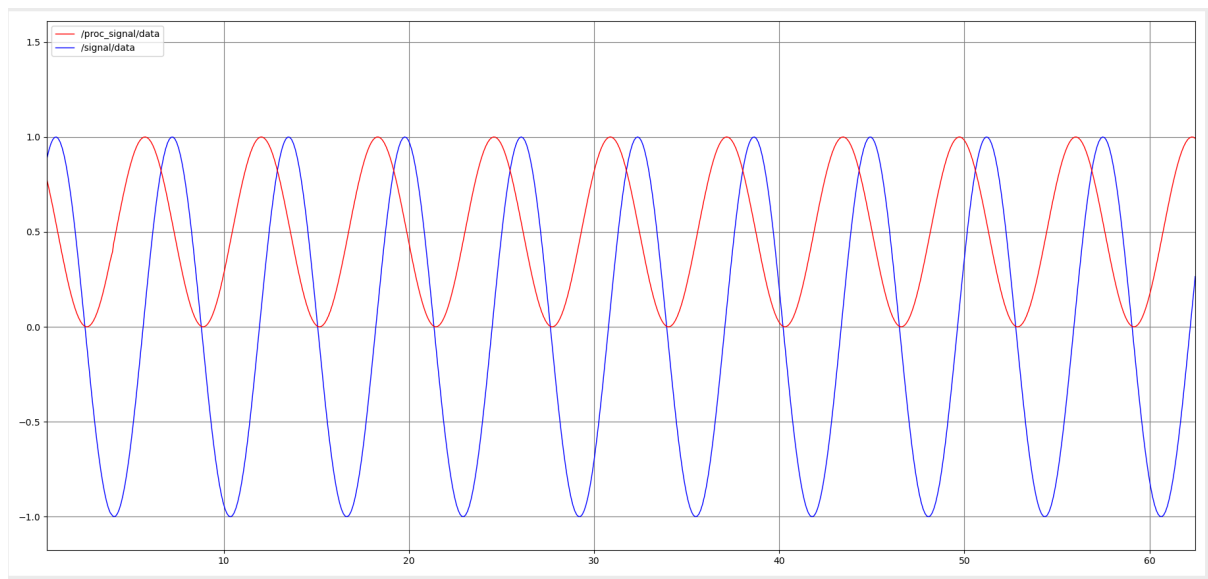
```
$ source install/setup.bash
```

```
$ ros2 launch signal_processing challenge1_launch.py
```

Nodos mostrados usando rqt_graph



Señales resultantes mostradas usando rqt_plot



En la gráfica se observan dos señales: `/signal/data` (azul), que es la señal original sinusoidal con amplitud aproximada entre -1 y 1, y `/proc_signal/data` (rojo), que corresponde a la señal procesada, la cual aparece desfasada respecto a la original y además fue desplazada y escalada para quedar en el rango aproximado de 0 a 1. Esto confirma que el nodo de procesamiento aplicó correctamente el desfase y la normalización, por lo que se cumplió el objetivo de generar, procesar y visualizar ambas señales en tiempo real.